VJTECH ACADEMY

Client Side Scripting (JavaScript)

Prepared As Per MSBTE I Schema Syllabus

Author:

"Prof. Vishal Jadhav"

[BE in Computer Engineering and Having 10 years of IT industry experience (Worked in Capgemini, Amdocs & currently working in TIBCO Software)]

UNIT 4 – Cookies and Browser Data

4.1 Cookies – basic of cookies, reading a cookie value, writing a cookie value, creating a cookies, deleting a cookies, setting the expiration date of cookie.

4.2 Browser – Opening a window, giving the new window focus, window position, changing the content of window, closing window, scrolling a web page, multiple windows at once, creating a web page in new window, JavaScript in URLs, JavaScript security, Timers, Browser location and History.

❖ JavaScript Cookies :

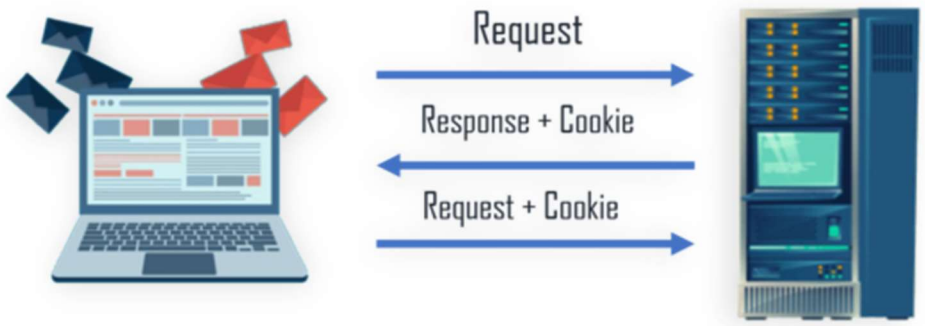
JavaScript Cookies are a great way to save a user's preferences in his / her browser. This is useful for websites and users both, if not used to steal privacy.

What Are Cookies?

- A cookie is a small piece of text stored on the visitor's computer by a web browser.
- As the information is stored on the hard drive it can be accessed later, even after the computer is turned off.
- They are primarily used for:
  - Session Management: Cookies can track user sessions, allowing users to stay logged in as they navigate a website.
  - Personalization: Websites can personalize content based on a user's past interactions and preferences.
  - Tracking: Cookies are often used for analytics and tracking user behavior.
- As cookies as a simple piece of text they are not executable.
- In JavaScript, we can create and retrieve cookie data with JavaScript's document object's cookie property.

How Cookies Work ?

- Cookies consist of key-value pairs and have various attributes:
  - **Name:** The name of the cookie.
  - **Value:** The data associated with the cookie.
  - **Expires:** The date and time when the cookie will expire.
  - **Domain:** The domain for which the cookie is valid.
  - **Path:** The URL path for which the cookie is valid.



Working Of Cookie

❖ Cookie operations in JavaScript

1. Creating Cookie :

- To create a cookie in JavaScript, you use the document.cookie property.
- Set this property to a string in the format "name=value", where "name" is the name of the cookie, and "value" is the data you want to store in the cookie.
- You can also specify additional attributes like expires, domain, and path to control when and where the cookie is valid.
- For Example :

```
document.cookie = "username=Sham; expires=Thu, 18 Dec 2023 12:00:00 UTC;
path=/";
```

- In this example, a cookie named "username" with the value "Sham" is created. It will expire on December 18, 2023, and is valid for all paths under the domain.

2. Reading Cookies :

- To read cookies, you can access the document.cookie property.
- Cookies are stored as a single string, with multiple cookies separated by semicolons and spaces.
- For Example :

```
var cookie_string = document.cookie;// it will return all cookies in string
format and it separates using semicolon
```

- You typically split the string to extract individual cookie values.
- For Example :

```
var cookie_array = document.cookie.split(';');
for(var i = 0; i < cookie_array.length; i++) {
    document.write(cookie_array[i] + "<br>");
}
```

- This code will split all cookies with semicolon and store in array and then print all cookies in new line.
- If there are no cookies, 'document.cookie' will return an empty string.

3. Updating Cookie [updating attributes] :

- To update a cookie, you essentially overwrite it by creating a new cookie with the same name.
- You can change the cookie's value, expiration date, domain, path, or any other attributes as needed.

1. Example of Updating a Cookie's Value:

```
// Create a cookie with an initial value
document.cookie = "username=Vishal";

// Update the value of the "username" cookie
document.cookie = "username=Vishal_Jadhav";
```

In this example, the "username" cookie is initially set with the value "Vishal," and then it's updated to have the value "Vishal_Jadhav" by creating a new cookie with the same name.

2. Example of Updating a Cookie's Expiration Date:

```
// set expiration date of cookie
var expiration_date = new Date();
expiration_date.setMonth(expiration_date.getMonth() + 1);
document.cookie = "username=Sham; expires=" + expiration_date.toUTCString() +
"; path=/";
// explanation -> this code will set expiration date of cookie to 1 month

// updating cookie expiration date by 2 months
var newExpiration_date = new Date();
newExpiration_date.setMonth(newExpiration_date.getMonth() + 2);
document.cookie = "username=Sham; expires=" + newExpiration_date.toUTCString()
+ "; path=/";
// explanation -> this code will update cookie expiration date by 2 months
```

4. Deleting a Cookie :

- To delete a cookie, you set its **expires** attribute to a date and time in the past. This tells the browser that the cookie has expired and should be removed.
- The date and time format should be in a UTC string format, such as **"Thu, 01 Jan 1970 00:00:00 UTC"**.

```
// Set the "username" cookie to expire immediately (a past date)
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

- In this example:
  - **"document.cookie"** is used to modify the cookie.
  - **"username="** is used to specify the cookie name.
  - **expires=Thu, 01 Jan 1970 00:00:00 UTC** sets the expiration date to January 1, 1970, which is in the past, causing the browser to delete the cookie.

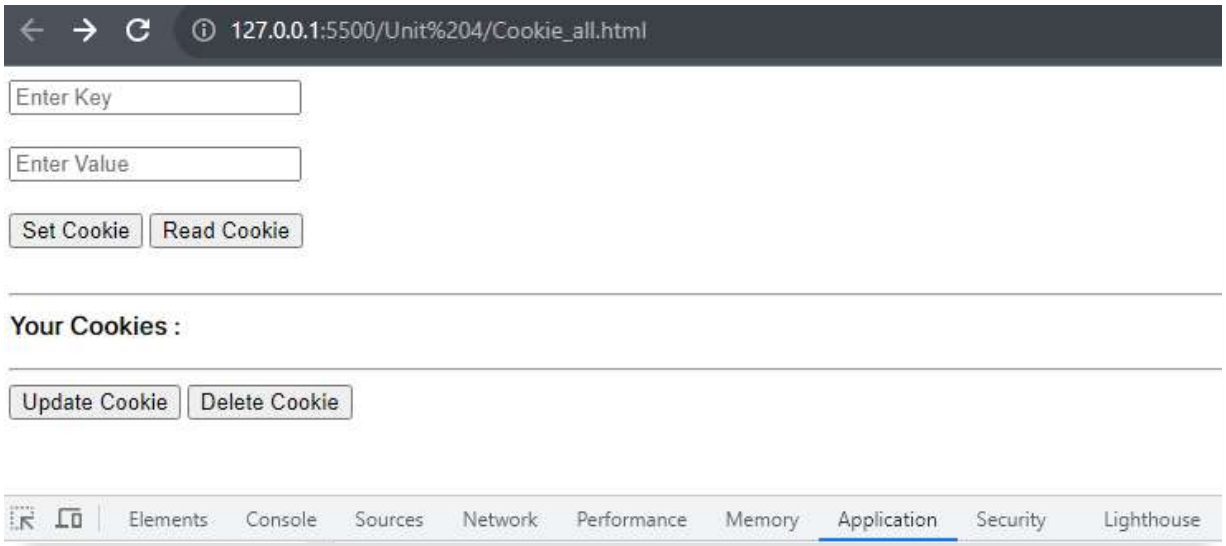
➢ Cookie Program [create, read, update, delete] :

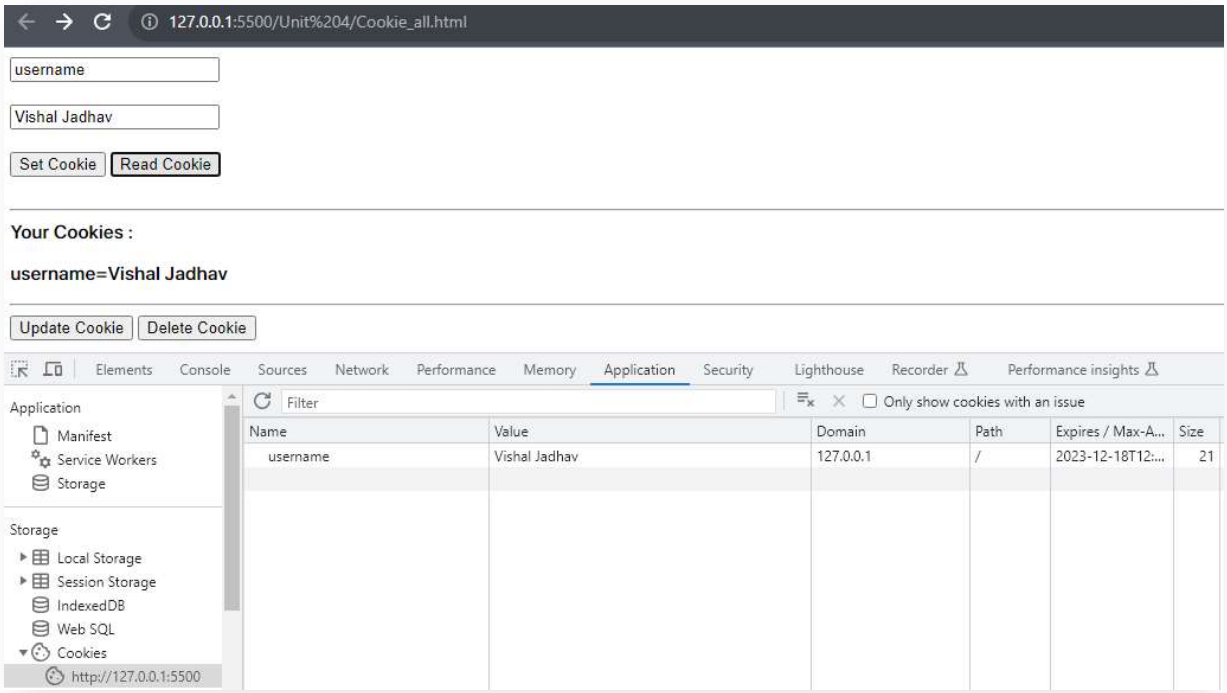
```
<html>
<head>
    <title>Cookie Example</title>
</head>
<body>
    <div>
        <input type="text" id="inputKey" placeholder="Enter Key">
        <br><br>
        <input type="text" id="inputValue" placeholder="Enter Value">
        <br><br>
        <input type="button" value="Set Cookie" onclick="setCookie()">
        <input type="submit" value="Read Cookie" onclick="readCookie()">
        <br><br>
        <hr>
        Your Cookies : <p id="myCookies"></p>
        <hr>
        <input type="button" value="Update Cookie" onclick="updateCookie()">
        <input type="button" value="Delete Cookie" onclick="deleteCookie()">
    </div>

    <script language="javascript" type="text/javascript">
        // set new cookie
        function setCookie() {
            var key = document.getElementById("inputKey").value;
            var value = document.getElementById("inputValue").value;
            document.cookie = key + "=" + value + "; expires=Thu, 18 Dec 2023
12:00:00 UTC; path=/";
            alert("Cookie Set Successfully")
        }
```

```
        // read cookie
        function readCookie() {
            if (document.cookie) {
                var cookie_array = document.cookie.split(';');
                var cookie_string = "";
                for (var i = 0; i < cookie_array.length; i++) {
                    cookie_string += cookie_array[i] + "<br>";
                }
                document.getElementById("myCookies").innerHTML = cookie_string;
            } else {
                document.getElementById("myCookies").innerHTML = "<i>No Cookies
Found</i>";
            }
        }
        // update cookie
        function updateCookie() {
            var key = prompt("Enter Key Of Cookie : ");
            var newValue = prompt("Enter New Value : ");
            document.cookie = "" + key + "=" + newValue + "; expires=Thu, 18 Dec
2023 12:00:00 UTC; path=/";
            alert("Cookie Updated : " + key + " : " + newValue);
        }
        // delete cookie
        function deleteCookie() {
            var key = prompt("Enter Key Of Cookie : ");
            document.cookie = "" + key + "=" + "; expires=Thu, 01 Jan 1970
00:00:00 UTC; path=/";
            alert("Cookie Deleted Where Key Is : " + key);
        }
    </script>
</body>
</html>
```

➢ Cookie Program Outputs :





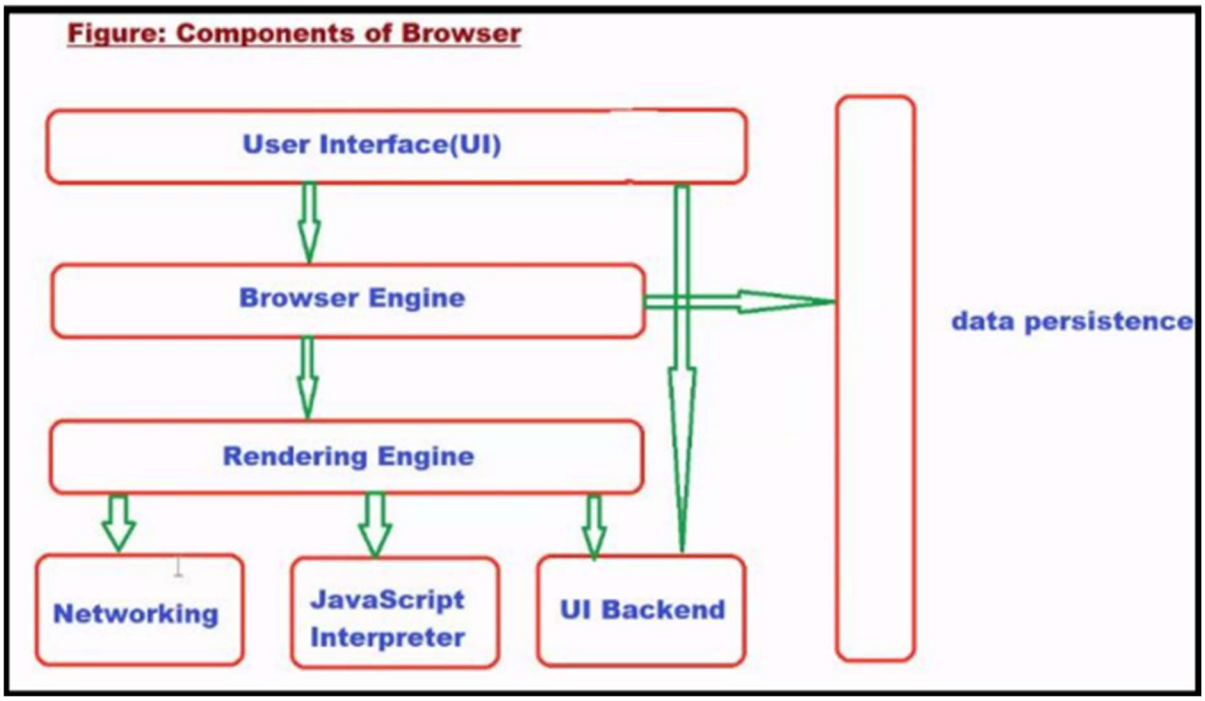
❖ JavaScript Browser Object Model (BOM) :

- The Browser Object Model (BOM) is a set of properties, rules, and methods in JavaScript that is used for containing all the information related to the browser and screen of a computer that we see at that instant of time. There is much information that we can fetch with the help of the **Window Object Model** in JavaScript Like,
- We can find the browser name that we are using currently, we can open new one or more at a time windows, we can find the dimension of that window screen, the page history like which page we are using before coming to the present page, etc.

❖ Methods Of Window Object :

Methods	Description
alert()	This method is used for displaying an alert box that consists of a message of the OK button.
clearInterval()	This method is used for clearing the time interval that has been set by the setInterval() method
close()	This method is sued for closing the current window that we are using.
confirm()	This method is used for displaying a dialogue box that shows two button options like OK and Cancel button
focus()	This method is used for setting the focus on the current window that we are using
open()	This method is used for opening a new window
moveTo()	This method is used for moving a particular window from one position to another position

moveBy()	This method is used for moving a window from the current place to the relative place
prompt()	This method is used for prompting an input
setTimeout()	This method is used for setting a timeout after which an expression will be evaluated
setInterval()	This method is used for setting a time interval for evaluating every expression.
scroll(x,y)	scroll(x,y) method scrolls the document by the specified number of pixels.
scrollBy()	scrollBy(x,y) method scrolls the document by the specified number of pixels.



❖ Code 1 :

```
<html>
<head>
    <title>Code 1</title>
</head>
<body>
    <script language="javascript" type="text/javascript">
        window.alert("This is VJTech Academy (alert)");
        window.confirm("Are You sure,do you want to delete the file? (prompt)");
        window.prompt("Enter Your Name: (prompt)");
    </script>
</body>
</html>
```

❖ Code 2 *"Opening New Window"* :

```
<html>
<head>
    <title>Open New Window</title>
</head>
<body>
    <button onclick="openNewWindow()">Open New Window</button>
    <script language="javascript" type="text/javascript">
        function openNewWindow() {
            window.open("https://www.vjtechacademy.in", "newWindow",
"width=300,height=300,toolbar=yes,scrollbars=yes,resizable=yes");
        }
    </script>
</body>
</html>
```

❖ **Code 3** ***"Get focus to window, close window, and blur opened window"*** **:**

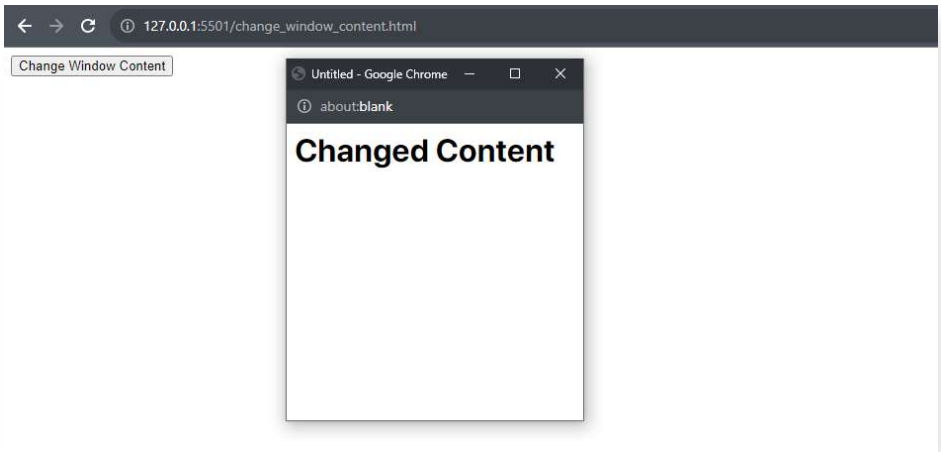
```
<html>
<head>
    <title>Focus, Blur & Close</title>
</head>
<body>
    <!-- 4 buttons for 1.Open window, 2.Close Window 3.Focus On Window 4.Blur
Window -->
    <button onclick="openVJTechAcademy()">Open VJTech Academy Site</button>
    <button onclick="closeWindow()">Close Site</button>
    <button onclick="focusWindow()">Focus Site</button>
    <button onclick="blurWindow()">Blur Site</button>
    <script language="javascript" type="text/javascript">
        var newWindow;
        function openVJTechAcademy() {
            newWindow = window.open("https://www.vjtechacademy.in", "newWindow",
"width=300,height=300,toolbar=yes,scrollbars=yes,resizable=yes");
        }
        function closeWindow() {
            newWindow.close();
        }
        function focusWindow() {
            newWindow.focus();
        }
        function blurWindow() {
            newWindow.blur();
        }
    </script>
</body>
</html>
```

❖ Code 4 *"Opening a window and after open change its content"*:

```
<html>
<head>
    <title>Changing Content Of Window</title>
</head>
<body>
    <!-- button to open window -->
    <button onclick="changeWindowContent()">Change Window Content</button>

    <script language="javascript" type="text/javascript">
        var newWindow;
        function changeWindowContent() {
            newWindow = window.open("https://www.vjtechacademy.in", "newWindow",
"width=300,height=300,toolbar=yes,scrollbars=yes,resizable=yes");
            newWindow.document.write("<h1>Changed Content</h1>");
        }
    </script>
</body>
</html>
```

Output :



❖ **Code 5** ***"add close button into newWindow and close opened window"*:**

```
<html>
<head>
    <title>New Window with Close Button</title>
    <script language="javascript" type="text/javascript">
        var newWindow;
        function openNewWindow() {
            newWindow = window.open("", "newWindow",
"width=300,height=300,toolbar=yes,scrollbars=yes,resizable=yes,left=200,top=200")
;
            newWindow.document.write("<p>This is a new window</p>");
            newWindow.document.write("<button
onclick='window.close()'>Close</button>");
        }
    </script>
</head>
<body>
    <button onclick="openNewWindow()">Open New Window</button>
</body>
</html>
```

❖ Code 6 *"open multiple windows at time"*:

```
<html>
<head>
    <title>Open Multiple Window</title>
</head>
<body>

    <button onclick="openMultipleWindow()">Open 5 Window</button>

    <script language="javascript" type="text/javascript">
        function openMultipleWindow() {
            for (let i = 0; i < 5; i++) {
                var myWindow = window.open("", "newWindow" + i,
"width=300,height=300,toolbar=yes,scrollbars=yes,resizable=yes,left=" + (i * 100)
+ ",top=" + (i * 100));
                // insert content into the new window
                myWindow.document.write("<p>This is a new window</p>"+(i+1));
            }
        }
    </script>
</body>
</html>
```

❖ Code 7 *"scroll webpage top to bottom"*:

```
<html>
<head>
    <title>Scroll To Bottom</title>
</head>
<body>
    <!-- button to scroll bottom -->
    <button onclick="scrollToBottom()">Scroll to bottom</button>
    <div id="multiple-line"></div>
    <script language="javascript" type="text/javascript">
        // add multiple lines in the div
        for (var i = 1; i <= 100; i++) {
            document.getElementById("multiple-line").innerHTML += "This is line "
+ i + "<br>";
        }
        function scrollToBottom() {
            window.scrollTo(0, document.body.scrollHeight);
        }
    </script>
</body>
</html>
```

"Note : here we can also use another different methods which are listed on our top table."

❖ JavaScript Timers :

Timers in JavaScript are used to schedule the execution of functions or code blocks at specific intervals or after a certain delay. Timers are crucial for creating animations, implementing timeouts, and handling asynchronous tasks. There are two main timer functions in JavaScript: setTimeout() and setInterval(). Here are detailed notes on how to use these timers effectively:

1. setInterval() Function:

- setInterval() method is used to execute a function repeatedly at a given time-interval.
- The setInterval() method will wait for a specified number of milliseconds, and then execute a specified function, and it will continue to execute the function, once at every given time-interval.
- The setInterval() method returns an ID value that can be used to refer to the timer created by the call to setInterval().
- The clearInterval() method is used to stop the timer.
- **Syntax :**

```
setInterval(function() {
    // Code to execute repeatedly
}, intervalInMilliseconds);
```

- **Parameters:**
  - The first argument is the function or code block to execute.
  - The second argument is the time interval (in milliseconds) between executions.

- **Example :**

```
// by using setInterval() method print VJTech Academy every 2 seconds
var counter=0;
var myInterval= setInterval(function() {
    document.write("VJTech Academy<br>");
    counter++;
}, 2000);
if(counter==10){
   clearInterval(myInterval);
}
```

2. setTimeout() Function:

- setTimeout() method is used to execute a function after a specified time.
- The setTimeout() method will wait for a specified number of milliseconds, and then execute a specified function.
- The setTimeout() method only executes the function once. If you need to repeat execution, use the setInterval() method.
- The setTimeout() method returns an ID value that can be used to refer to the timer created by the call to setTimeout().
- The clearTimeout() method is used to stop the timer.
- **Syntax :**

```
setTimeout(function() {
    // Code to execute after the delay
}, delayInMilliseconds);
```

- **Parameters :**
  - The first argument is the function or code block to execute.
  - The second argument is the delay in milliseconds before execution.

- **Example :**

```
// print VJTech Academy after 5 seconds
var myTmOut = setTimeout(function () {
    document.write("VJTech Academy");
}, 5000);
// stop the timer before the function is executed using clearTimeout() method
clearTimeout(myTmOut);
```

❖ Window History :

- We can also get the browser history by using the window.history property in the window object in JavaScript.
- The window.history property helps us to access the information about the pages that we visited earlier from that current page. Also, this should not be confused with the API of the new HTML5 history. These are different.
- We can also get various information related to the history of a web page using different methods like -
  - window.history.length: This property returns the number of entries in the browsing history. It indicates how many pages the user has visited in the current session.
  - window.history.go():The go() method allows you to navigate to a specific page in the browsing history by specifying a positive or negative integer. A positive integer goes forward, while a negative integer goes backward.

```
window.history.go(1); // goes forward by 1 page
window.history.go(0); // current page will be reloaded
window.history.go(-1); // goes back by 1 page
```

  - window.history.forward() and window.history.back(): These two methods of window.history property can be used to navigate through one page forward or one page backward respectively.

- **Example :**

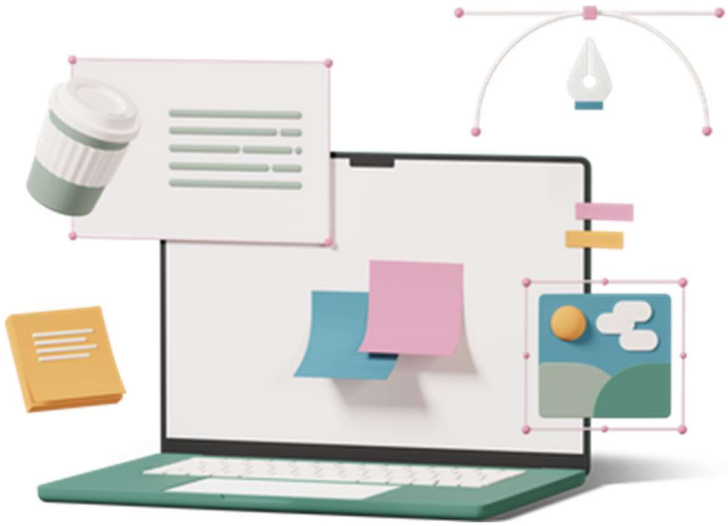
```
<html>
<head>
    <title>Window History</title>
</head>
<body>
    <!-- add 2 web links to visit  -->
    <a href="http://www.vjtechacademy.in">VJTech Academy</a><br>
    <a href="http://www.google.com">Google</a> <br>
    <!-- add 3 buttons to go back, go forward and go to the specified page -->
    <button onclick="goBack()">Go Back</button>
    <button onclick="goForward()">Go Forward</button>
    <button onclick="goTo()">Go To</button>
    <script language="javascript" type="text/javascript">
        // go back to the previous page
        function goBack() {
            window.history.back();
        }
        // go forward to the next page
        function goForward() {
            window.history.forward();
        }
        // go to the specified page after visit 2 web links
        function goTo() {
            window.history.go(2);
        }
    </script>
</body>
</html>
```

❖ Window Location :

- The window.location object represents the current URL of the browser. It provides information about the current page's URL and allows you to navigate to other URLs. Here are key points about window.location :
  - location.href : This property contains the complete URL of the current page, including the protocol (http/https), domain, path, query parameters, and fragment identifier.
  - location.hostname : This property returns the domain name of the current URL (e.g., "example.com").
  - location.pathname : This property returns the path part of the URL (e.g., "/products/page1.html").
  - location.hash : This property contains the fragment identifier of the URL (e.g., "#section1").
  - window.location.reload(): Reloads the current page.

- **Example :**

```
<html>
<head>
    <title>Window Location</title>
</head>
<body>
    <script language="javascript" type="text/javascript">
        location = location.href;
        document.write("<h1>Href : " + location.href);
        document.write("<br>Protocol : " + location.protocol);
        document.write("<br>path : " + location.pathname);
        document.write("<br>Host : " + location.host);
    </script>
</body>
</html>
```



VJTECH ACADEMY

CLASS NOTES